

AI 엔지니어링의 4가지 축

Prompt와 Context만으로는 강한 에이전트를 안전하게 운용할 수 없습니다.
Harness는 그 사이의 빈칸을 메우는 통제 시스템입니다.

01

Prompt Engineering

무엇을 원하나를 정확히 말하는 기술

02

Context Engineering

작업에 필요한 정보를 제대로 공급하는 기술

03

Harness Engineering

규칙, 게이트, 경계, 실행 루프를 설계하는 기술

04

Agentic Engineering

역할을 나누고 작업을 오케스트레이션하는 기술

Prompt → Context 다음에 필요한 건 통제입니다

정보가 많아질수록 더 잘 이해할 수는 있어도, 더 안전하게 행동하는 건 아닙니다.

STAGE 01

Prompt Engineering

“무엇을 만들어줘?”를 정확하게 말한다.

예: 결제 API 연동해줘



STAGE 02

Context Engineering

기술 스택, 코드 구조, 정책 문서를 제공한다.

이해를 늘린다



STAGE 03

Harness Engineering

DB 직접 호출, 민감정보 출력, 규칙 우회를 시스템 차원에서 막는다.

문제는 정보 부족이 아니라 통제 부족입니다

AI가 맥락을 다 알아도 잘못된 방향으로 갈 수 있습니다. 그래서 하네스는 “알게 하는 것”보다 “못 하게 막는 것”에 집중합니다.

● 정보만 제공

모든 걸 알고 있어도 위험할 수 있다

지식은 판단 재료일 뿐입니다. 경계가 없으면 잘못된 행동은 그대로 실행됩니다.

- 스키마를 임의로 바꾼다
- 민감한 데이터를 로그에 남긴다

● 하네스로 통제

정보와 통제를 분리해 설계한다

무엇을 아는가와 무엇을 할 수 있는가는 다른 문제입니다. 하네스는 실행 권한과 범위를 구조적으로 제한합니다.

- 테스트 게이트를 통과해야만 다음 단계로 이동

AI 에이전트는 강한 말이고, 하네스는 그 힘을 다루는 마구입니다

에이전트는 엄청난 힘을 가진다

코드를 쓰고, 도구를 호출하고, 파일을 읽고, 작업을 연쇄적으로 확장할 수 있습니다.

- 복잡한 작업을 빠르게 밀어붙인다
- 지시가 넓으면 예상보다 멀리 나간다
- 반복을 자동화할수록 영향 범위가 커진다

통제 없이는 힘이 곧 위험이 된다

강할수록 더 세밀한 제어가 필요합니다. 방향, 속도, 한계가 없으면 같은 실수가 더 빠르게 반복됩니다.

- 가드레일은 방향을 정한다
- 테스트 게이트는 속도를 조절한다
- 도구 경계는 넘어가면 안 되는 선을 만든다

하네스의 4가지 기둥

1

기계가 읽는 규칙 파일

AGENTS.md, CLAUDE.md, 정책 문서처럼 런타임에 주입되는 명시적 규칙이 있어야 합니다.

2

결정론적 게이트

린트, 구조 테스트, pre-commit, CI 같은 품질 관문이 AI 출력 앞에 서 있어야 합니다.

3

명시적 도구 경계

파일, API, DB, 배포 권한을 분리해서 접근 가능한 범위를 의도적으로 좁혀야 합니다.

4

지속적 피드백 루프

실패를 분석하고 규칙에 반영해, 같은 오류가 다음 실행에 서는 다시 못 나오게 만들어야 합니다.

하네스 시스템은 네 단계로 작동합니다

1

분류기

요청이 단순 질문인지, 실제 실행이 필요한 작업인지 먼저 구분합니다.

2

컨텍스트 관리자

필요한 파일과 규칙만 선별해서 주입하고, 과도한 정보는 오히려 제거합니다.

3

실행 루프

테스트를 통과할 때까지 자동 반복하되, 실패 원인은 시스템 규칙으로 환원합니다.

오늘 하나만 기억하면 됩니다

1

AI 에이전트가 실수했을 때,
프롬프트를 먼저 고치지 마세요.
마구를 고치세요.

그 실패가 구조적으로 바보 분가능하도록